Sentinel AI

Real-Time Scam Protection System

Prepared by	Sentinel AI Project Team
Project Type	Android-based real-time phishing and scam protection system
Primary Platform	Android 8.0 (API 26) and later
Date	July 21, 2026

A technical project report covering system design, machine learning workflow, implementation, privacy model, evaluation, challenges, and future development scope.

Abstract

Sentinel AI is a real-time scam protection system designed to reduce phishing and social-engineering risk at the moment a user is most vulnerable: before opening a suspicious link or responding to a deceptive notification. Phishing attacks commonly imitate trusted institutions, hide unsafe destinations, and use urgency to pressure users into sharing credentials, payment details, or personal information. Sentinel AI addresses this problem through an Android-first, local protection pipeline that intercepts links, normalizes input, extracts risk signals, runs explainable heuristic checks, and applies an on-device TensorFlow Lite URL classifier. The system combines evidence into an understandable ALLOW, WARN, or BLOCK decision with a risk score and user-facing reasons. Its key innovation is click-time prevention with offline machine learning and privacy-preserving analysis: URL features, notification content, model inference, preferences, and scan history remain on the device by default. The result is a practical defense layer that helps users make safer decisions without relying on a cloud backend or exposing sensitive content.

1. Introduction

Phishing and digital scam attacks have become one of the most common ways users lose account access, money, and personal data. Unlike attacks that depend entirely on technical software vulnerabilities, phishing frequently succeeds by manipulating human behavior. Attackers create realistic messages, impersonate banks or public agencies, and direct users to fraudulent pages that request passwords, one-time codes, payment details, identity documents, or wallet credentials.

Mobile users face this risk in a compressed decision environment. Links arrive through messaging apps, email clients, social platforms, browser redirects, and notifications. The destination is often shortened, encoded, or visually similar to a trusted brand. Existing protections may warn after a site is loaded, depend on external blocklists, or fail when a new phishing campaign has not yet been reported. A real-time prevention layer is therefore needed before navigation occurs.

Sentinel AI is built around this prevention-first idea. It adds a decision point between user intent and browser handoff, while also scanning supported notifications for scam-like language and unsafe URL patterns. The system is intended to convert technical risk signals into clear action guidance.

2. Problem Statement

Users are regularly exposed to links and messages that imitate legitimate organizations and attempt to steal credentials, payments, or personal information. The problem is important because a single unsafe click can lead to financial loss, account takeover, identity exposure, malicious downloads, and continued impersonation of trusted contacts.

Existing protection methods have several limitations. Browser warnings may appear only after navigation has started. Server-side reputation services can miss newly created phishing domains and may require sharing the URL externally. General antivirus tools often focus on files rather than intent-time link decisions. Manual user judgment is unreliable when the attacker uses urgency, brand imitation, shortened URLs, raw IP addresses, or familiar sender names.

The central problem addressed by Sentinel AI is how to assess a link or notification quickly, privately, and explainably before the user proceeds with a risky action.

3. Proposed Solution

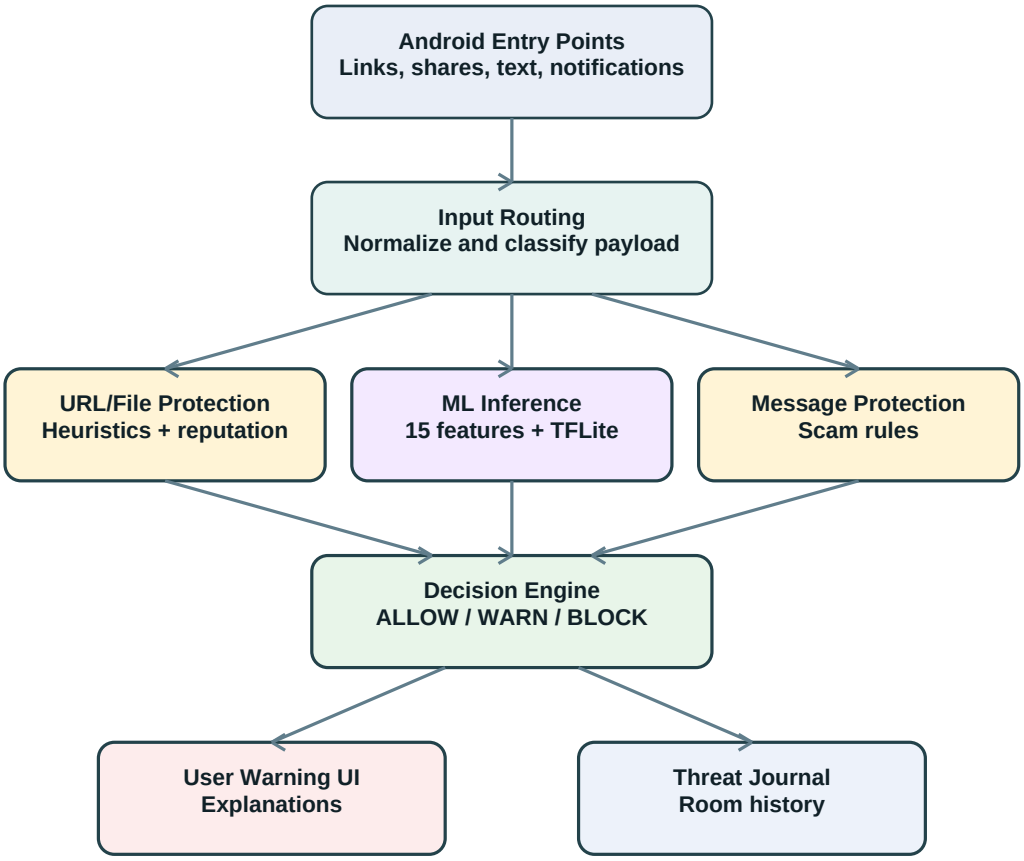
Sentinel AI proposes a local-first Android protection pipeline. When a user opens, shares, selects, or manually scans a URL-like input, the app captures the payload, normalizes it, analyzes it, and returns a decision before the browser or target application receives the link. For supported notifications, the app locally evaluates message text, extracted URLs, urgency indicators, credential-related terms, financial language, sender context, and duplicate events.

The decision system is risk-based. It does not expose raw technical features as the final user experience. Instead, it combines URL heuristics, optional reputation evidence, and on-device model output into a score and action. ALLOW indicates no strong local or provider evidence, WARN indicates meaningful suspicious signals, and BLOCK indicates a critical local score or malicious reputation evidence.

This design creates a practical middle layer between user intent and unsafe action. It remains usable offline for core protection and does not require a Sentinel AI backend.

4. System Architecture

Sentinel AI is structured as a multi-module Android application. The app module handles startup, intent routing, URL and file orchestration, machine learning inference, reputation integration, and warnings. The core module defines domain models, validation, event handling, Room storage, networking contracts, and feature state. The agents module performs notification parsing, supported-app routing, message event construction, and scam rules. Services run guard and monitoring work, while the UI module contains Compose screens, navigation, view models, scanner, dashboard, alert, detail, settings, and history interfaces.



The architecture begins with Android entry points such as web intents, share intents, selected text, manual scanner input, and notification listener events. Inputs are routed and normalized before moving into either URL/file protection or message protection. URL protection applies local heuristics, optional reputation checks, and the TensorFlow Lite model. Message protection applies notification-specific parsing and scam rules. Results flow through a threat event bus to the warning UI and local threat journal.

The ML inference flow transforms a normalized URL into 15 structural and lexical features, standardizes them with bundled scaler values, executes a float TensorFlow Lite model with input shape [1, 15], and produces a phishing probability. The evidence layer remains responsible for the final ALLOW, WARN, or BLOCK action, while the model contributes to the reported score.

5. Machine Learning Model

The machine learning component estimates phishing probability from engineered URL features. It complements the heuristic and reputation layers rather than replacing them. The training corpus described by the project contains more than 238,000 labeled URLs for binary classification of benign and phishing-like URLs. The deployed Android artifact is model.tflite, packaged with a matching scaler.json file.

No.	Feature	Purpose
1	URLLength	Total normalized URL length
2	DomainLength	Hostname length

3	IsDomainIP	Detects raw IPv4 hostnames
4	NoOfSubDomain	Counts extra hostname labels
5	IsHTTPS	Identifies HTTPS scheme usage
6	HasSuspiciousWords	Flags terms such as login, verify, account, or crypto
7	SpecialCharRatio	Measures non-alphanumeric character density
8	DigitRatio	Measures digit density
9	HasAtSymbol	Detects @ symbols in URLs
10	SuspiciousTLD	Flags configured suspicious top-level labels
11	BrandImpersonationScore	Combines brand, suspicious-word, and hyphen signals
12	HyphenCount	Counts hyphens in the URL
13	PathQueryLength	Measures combined path and query length
14	KnownBrandDomain	Checks official brand-domain matches
15	DomainVowelRatio	Measures vowel density in hostname

The training approach follows a standard TensorFlow pipeline: clean and label URLs, extract the same 15 features used by Android, fit feature means and scales on the training partition, train a binary classifier using held-out validation and test partitions, export the trained model to TensorFlow Lite, and package the model with matching scaler values. At runtime, Android validates the expected model input and output tensor shapes to prevent incompatible assets from being used silently.

The reported score blends model probability with evidence-based scoring. The model probability is converted to a percentage, boosted, clamped, and combined using a 70 percent evidence score and 30 percent boosted ML score formula. This keeps the model influential while preserving rule-based blocking for clear high-risk evidence.

Sample URL	Model phishing probability	Interpretation
google.com	0.00002	Safe or near-zero risk signal
shipaton.com	0.07	Low but non-zero suspicious signal
secure-login-google.xyz	1.00	High-confidence phishing-like signal
example.com/login	0.49	Mid-range suspicious signal requiring other evidence

6. Features of the System

- **Click-time protection:** Intercepts HTTP and HTTPS links before browser handoff and can block unsafe navigation.
- **Notification scanning:** Reviews supported messaging, email, and social notifications for URLs, urgency, financial language, credential requests, and sender context.
- **Risk scoring:** Converts local and provider evidence into GREEN, YELLOW, RED, or CRITICAL score bands and ALLOW, WARN, or BLOCK actions.
- **Manual scanning:** Provides in-app scanning for links, text, and supported file references.

- **Offline functionality:** Local heuristics, notification rules, file heuristics, and TFLite inference continue without cloud services.
- **Privacy-first design:** Scan history, preferences, URL features, and message analysis remain on the device by default.
- **Local history:** Threat events are stored in a private Room database and shown in dashboard, alert, detail, and history views.

7. Implementation Details

The Android implementation uses intent filters to receive web-open actions, shared links, selected URL-like text, and supported file references. IntentRouterActivity checks whether click protection is enabled, identifies the incoming payload, and routes it toward loading and scanning screens. ScanRepository coordinates heuristic analysis, reputation evidence, and ML inference before a result is displayed.

The ML path is managed through bundled application assets. model.tflite contains the TensorFlow Lite classifier, while scaler.json stores the feature means and scales required for standardization. The app extracts raw URL features in the documented order, transforms them into a Float32 tensor, invokes the interpreter, and blends the resulting probability with evidence-based scoring.

For notifications, SentinelNotificationListener receives events only after the user grants notification-listener access. NotificationAgentCoordinator parses the sender and message text, normalizes the event, extracts URL and content signals, suppresses near-duplicate notifications, and emits elevated events to the warning notification path and local journal.

The application uses Hilt for dependency injection, coroutines and flows for asynchronous state, and Room for persistent local threat history. Feature toggles are stored in private Android preferences and include real-time protection, notification protection, click protection, and text-selection analysis.

8. Privacy and Security Considerations

Sentinel AI is designed so the primary detection paths run on the Android device. In the default repository configuration, message content, file content, URL feature vectors, and scan history are not sent to a Sentinel AI backend. The app does not require an account or cloud inference service for its core protection.

The TensorFlow Lite model and scaler are packaged with the application, which enables offline inference, low latency, a fixed model version per build, and reduced exposure of browsing and message data. Scan records are stored in the app private Room database, while preferences are stored in private Android preferences.

The repository supports optional reputation integrations. OpenPhish can download a feed and compare scanned URLs locally; VirusTotal remains disabled unless a developer supplies an API key. If VirusTotal is enabled, scanned URLs are submitted externally and that configuration must be disclosed. For strict offline use, external reputation providers can be disabled while retaining local heuristics and ML inference.

9. Challenges Faced

- **Dataset balancing:** Phishing and benign URL datasets must be balanced carefully so the model does not overfit one class or produce misleading confidence.
- **Feature extraction consistency:** The Android app and training pipeline must preserve the exact same 15-feature order, data types, and scaler values.
- **Malformed URL handling:** The runtime must generate a finite feature vector even when inputs are incomplete, unusual, or intentionally malformed.
- **Real-time performance:** Click-time protection must analyze a link quickly enough that the user experience remains smooth.
- **Android ML integration:** The TFLite model, scaler, tensor shapes, asset loading, and lifecycle management must work reliably across devices.
- **Privacy boundaries:** Optional reputation checks must be clearly separated from offline inference so user data handling remains transparent.

10. Results and Evaluation

The project demonstrates a layered protection system that can evaluate URLs at click time, scan supported notifications, produce risk scores, and keep history locally. The example model predictions show near-zero probability for clearly safe domains, mid-range values for ambiguous login paths, and high confidence for brand-impersonation style phishing domains.

The project target and reported demonstration accuracy are approximately 99 percent for the URL classification task. Because the repository contains the deployed inference model and scaler but not the original training dataset, training scripts, or versioned evaluation report, this figure should be treated as a project-level reported result rather than a reproducible benchmark from the current repository alone.

Model reliability is strengthened by using the classifier as one part of a layered decision system. Heuristic rules and provider evidence can still warn or block even when the model score is inconclusive, while unknown or failed reputation lookups do not reduce local risk. This avoids presenting missing evidence as safety.

11. Future Scope

- Evaluate lightweight deep-learning and sequence models for more complex URL patterns while preserving on-device latency.
- Expand scam-text detection for Indian languages and transliterated message content.
- Add browser integrations for Chrome, Edge, Firefox, and other navigation surfaces.
- Extend support to iOS and desktop platforms where platform APIs permit similar protection flows.
- Add domain-age, certificate, redirect-chain, lookalike-domain, internationalized-domain, QR-code, and document-link signals.
- Introduce versioned model evaluation reports with accuracy, precision, recall, F1-score, latency, and model-size comparisons.

12. Conclusion

Sentinel AI achieves a practical real-time defense layer against phishing and scam attempts on Android. It intercepts risky user actions before navigation, analyzes links and notifications locally, applies both explainable rules and on-device machine learning, and presents simple decisions that users can act on immediately.

The project matters because phishing prevention is most effective before a user enters credentials, sends money, or opens a malicious destination. By combining click-time interception, local inference, risk scoring, notification analysis, and privacy-first storage, Sentinel AI demonstrates a submission-ready approach to safer mobile interaction without depending on a central backend.